

让编剧成为程序员： 基于故事语言的叙事灰盒化

融合剧本语义与代码逻辑的独立迭代工作流

SCENE 1: THE DEVELOPMENT STUDIO - DAY

#ACT_I INT. STUDIO - DAY

#SCENE_BREAK A lone CREATIVE WRITER sits at a desk, fingers poised over a keyboard.

#SCENE_BREAK

WRITER
(to self)

How do I translate this emotion into a narrative needs to branch on the player's choice.

[K_START]
Path ahead is uncertain."

chooses between left and

[K_END]

#ACT_I

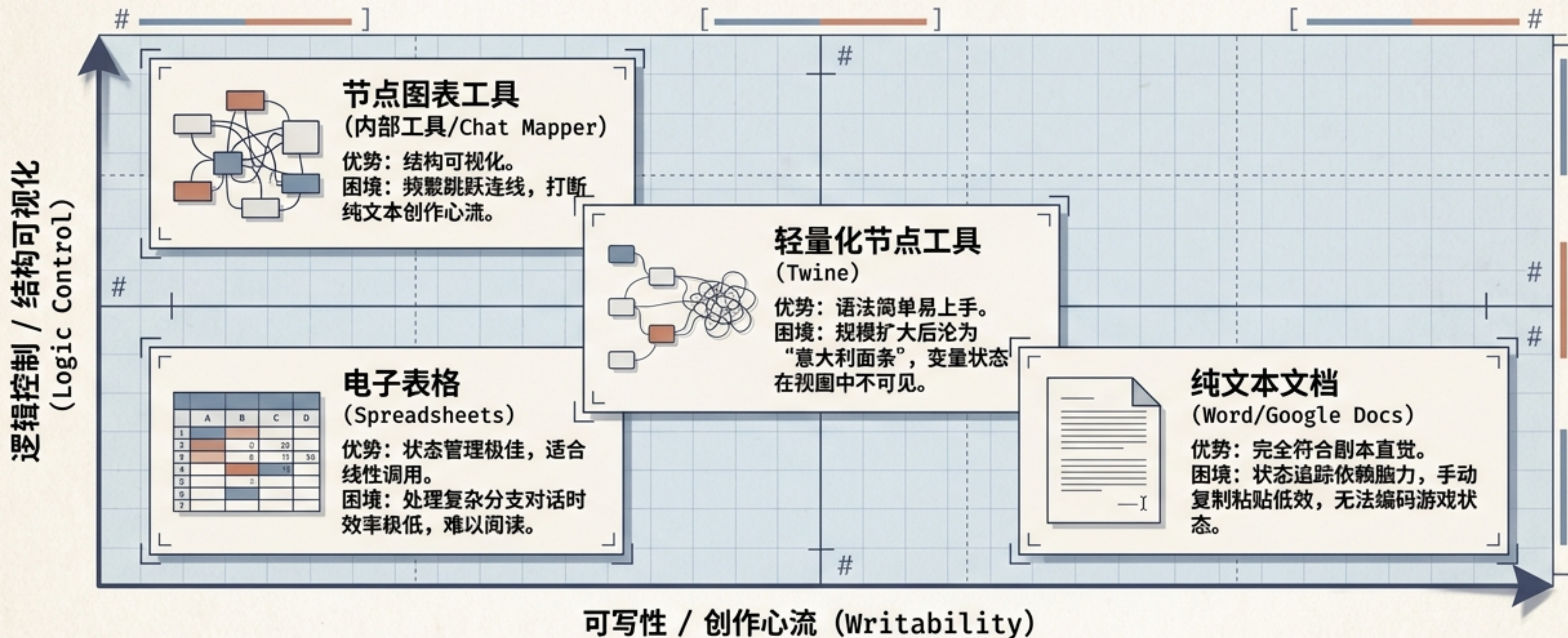
```
[#CODE_SNIPPET]
if (player.choice == 'left') {
    trigger(SCENE_02_FOREST);
} else {
    trigger(SCENE_03_MOUNTAIN);
}
[/#CODE_SNIPPET]
```

#DIALOGUE_TAG

WRITER

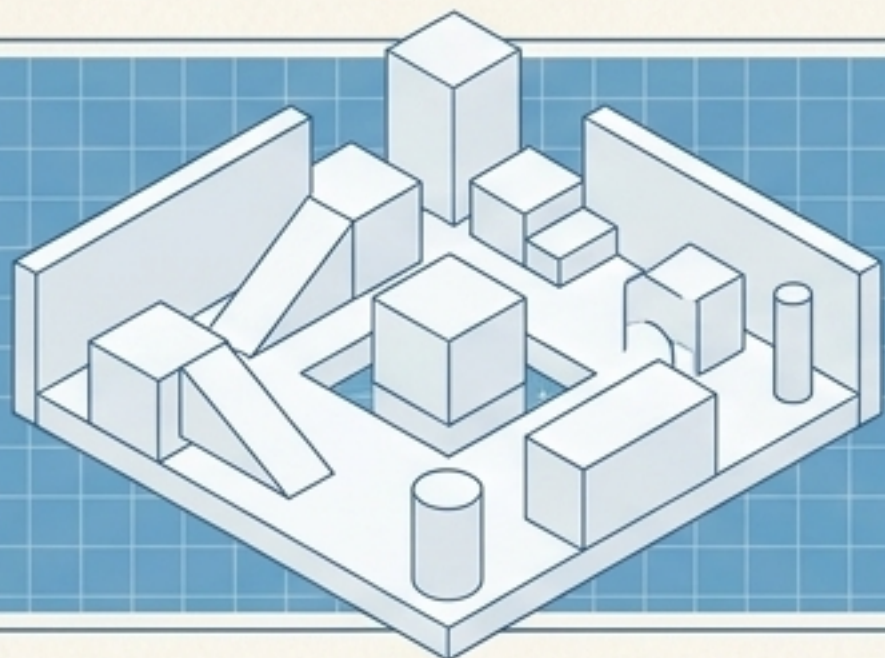
It's... a script that thinks."

传统工具的割裂：在“可视化”与“可写性”之间妥协



当前生态迫使设计者在“友好的剧本格式”与“开发者友好的逻辑控制”之间二选一。

核心理念：将场景逻辑等同于场景故事

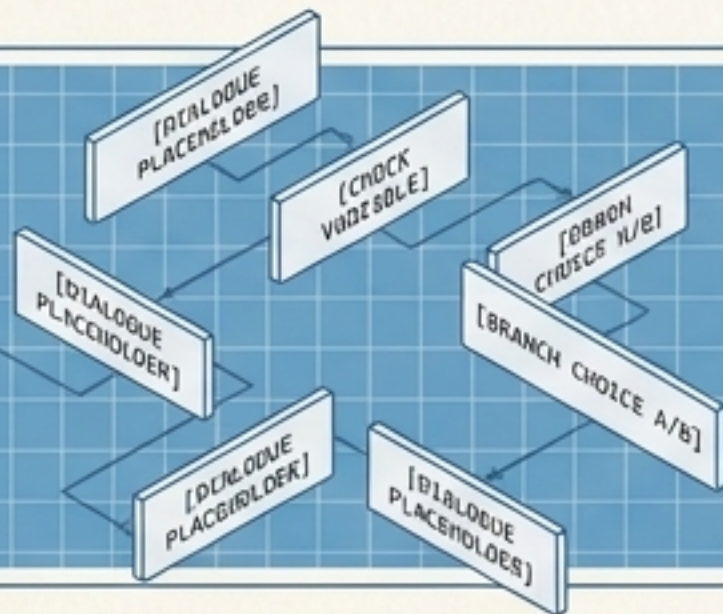


【类比】关卡灰盒化 (Level Greyboxing)

关卡设计师在投入昂贵美术资产前，先用基础几何体构建场景轮廓，测试空间与动线。

赋予编剧控制权：自主搭建迷你系统，无需等待底层支持。

程序员逆向赋能：提前构建系统脚手架填入占位符，后续无缝替换。

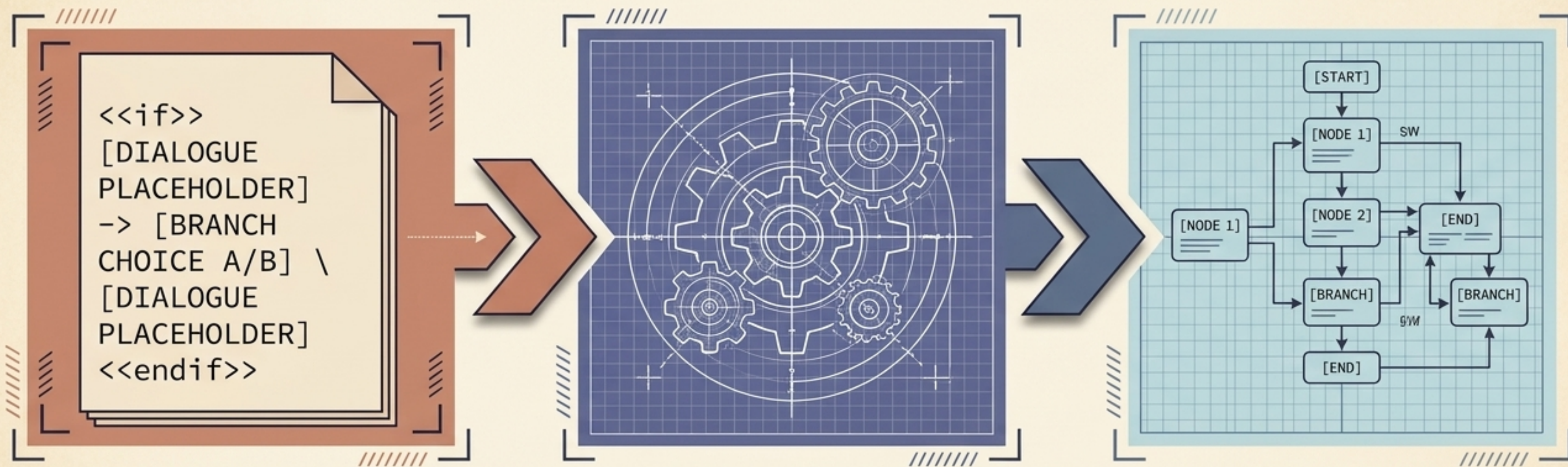


【转化】叙事灰盒化 (Narrative Greyboxing)

逻辑结构的搭建就是故事的搭建。在叙事游戏中，谁在做什么、说什么，就是游戏逻辑本身。

打破“先写文案，再配逻辑”的线性工作流，实现逻辑与内容的同步实验。

跨界融合：作为编程语言的剧本格式



编剧在纯文本编辑器中编写自上而下阅读的剧本，穿插轻量级控制语法。这是介于剧本与逻辑图的中间态。

引擎解析 (Parser)

引擎/编辑器工具读取纯文本文件，将其自动解析并转换为底层图形节点 (Nodes)。

游戏运行 (Gameplay Loop)

系统在节点间移动，发送对话、展示分支选项、执行后台命令，并依据状态跳转。

最小化语法：用字符串驱动复杂系统

```
<<if $angus_investigation == 0>>  
  <<set $angus_investigation to 1>>  
<<endif>>  
<<close_all>>  
Angus: 我们有麻烦了。 #loc_1042  
-> 询问情况  
  Angus: 激光网被触发了。  
-> 保持沉默  
  <<play_animation "nervous">>
```

[变量与控制]

像程序员一样管理状态，例如追踪角色的调查任务进度。

[引擎命令]

括号内的命令是“不向玩家展示的字符串”。编剧预演画面，直接写入控制动画的占位命令。

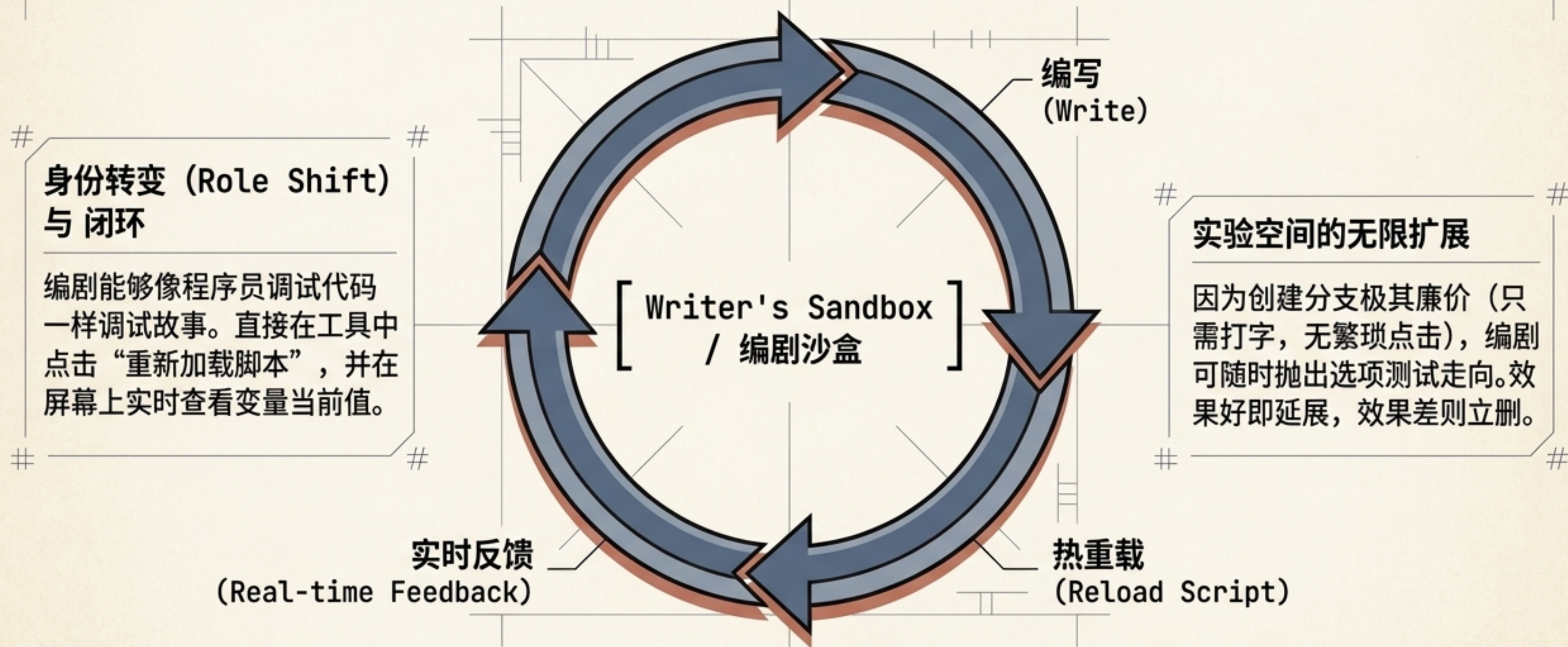
[标签与数据附加]

无需玩家可见的数据（如本地化代码）只需作为哈希标签附加在行末，避免复杂的表单录入。

[极简分支]

因果：无需新建节点并鼠标连线。创建分支的时间成本趋近于零，彻底解放编剧探索欲。

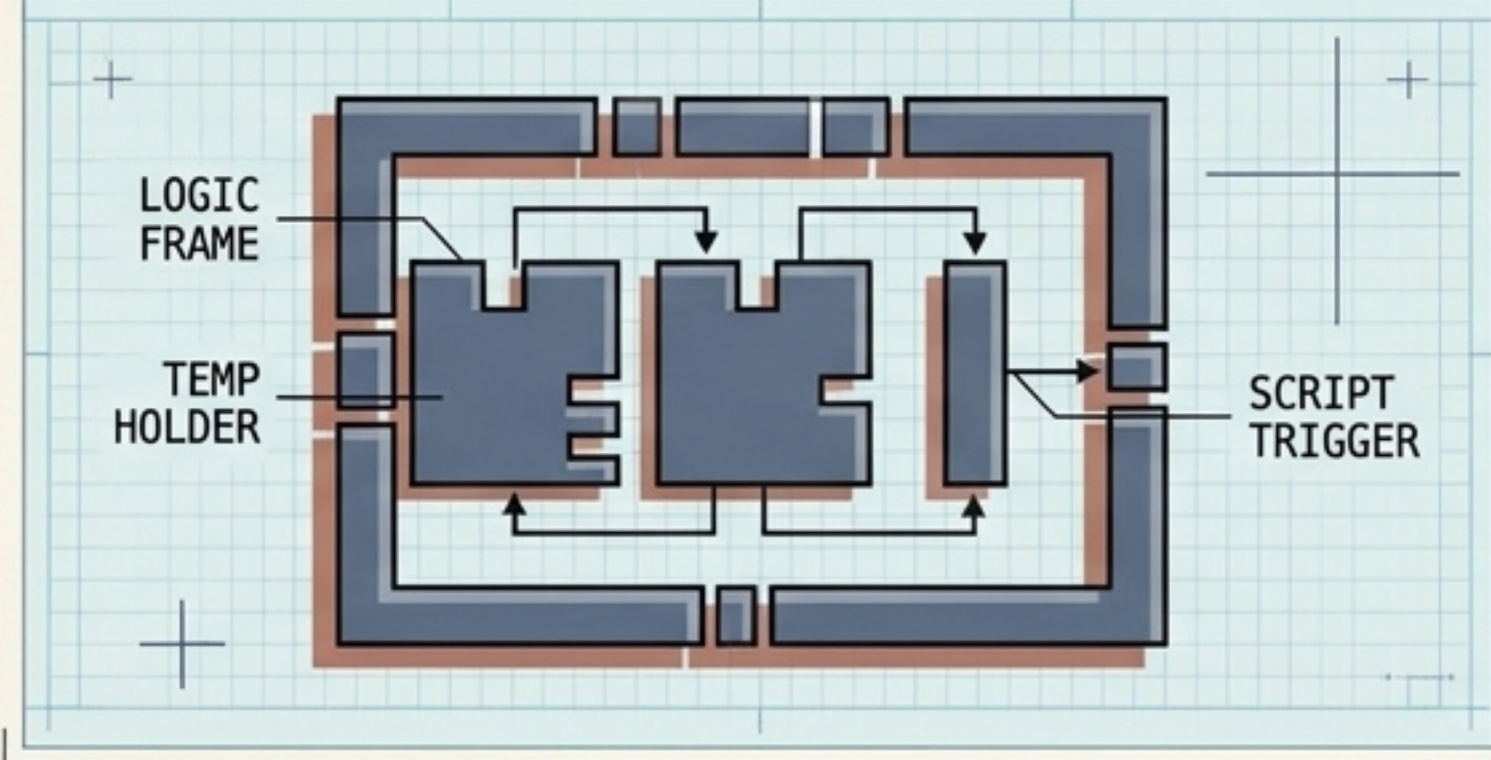
极速原型：低成本试错的沙盒效应



核心影响：基于纯文本的热重载机制，将叙事开发的重心从“与笨重的工具作斗争”彻底回归到“高频迭代故事本身”。

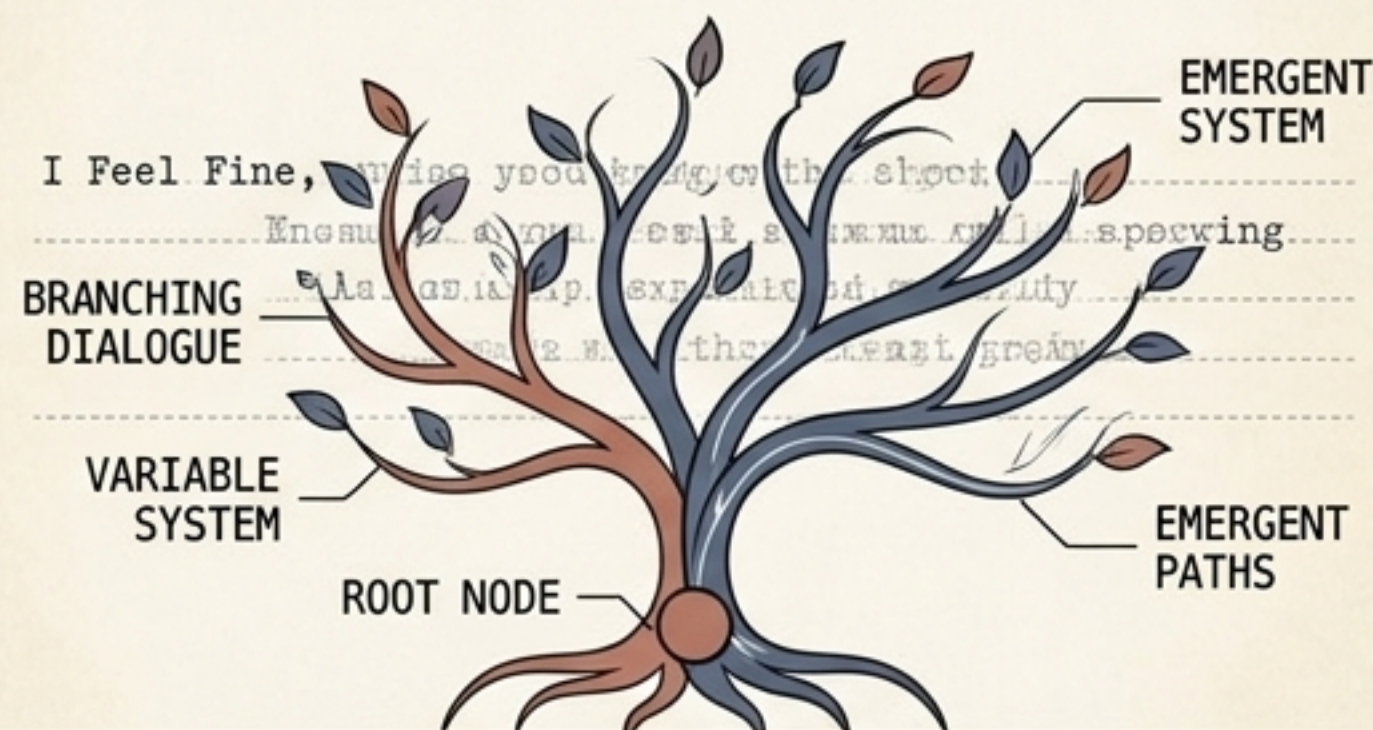
实践验证：从占位框架到有机系统生成

Night in the Woods (自上而下的灰盒化)



- 情境：游戏中所有过场动画均由脚本驱动。
- 应用：结局场景在极早期就完成了“逻辑占位(Temped out)”。逻辑结构贯穿开发周期保持不变。
- 最终真实对话内容直至开发最后期才由编剧填入。完美解决通用线性内容交付。

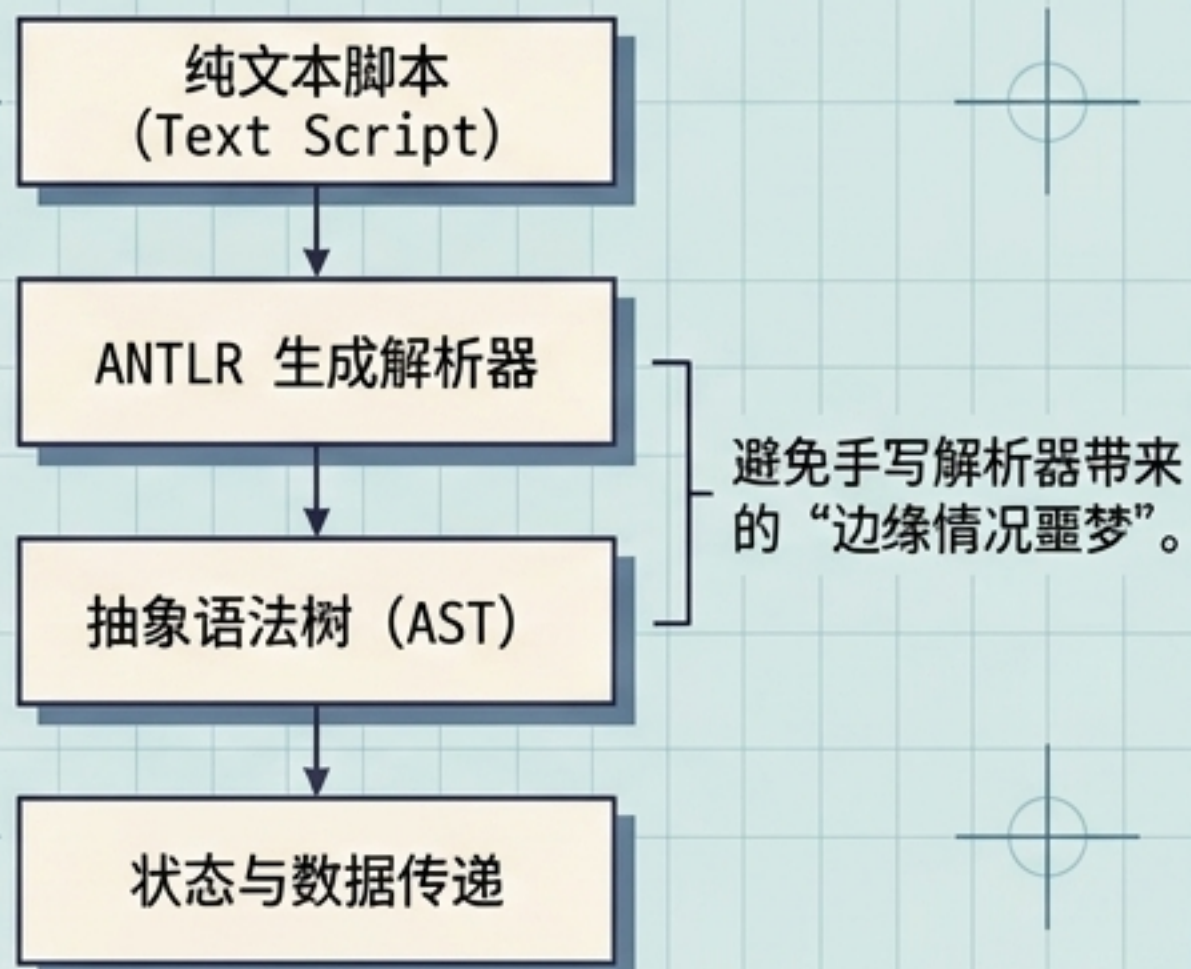
I Feel Fine (自下而上的系统涌现)



- 情境：最初提案仅为“单一房间、角色固定”的5分钟Demo，3天内扩展为1小时完整流程。
- 应用：编剧利用基础交互控制，在未提前规划下，自发构建了基于变量的“多角色声望系统”。
- 给予沙盒，复杂的底层交互系统便会在写作中自然“涌现”。

克制的工程哲学：不试图取代 C#

解析器架构 (Parser Architecture)



设计的取舍 (Design Trade-offs)

A. 悬置问题 (Punting Questions)

不需要解决所有问题。“每一行需要什么数据？”
——直接发送整条字符串，由游戏端自由解析。

B. 功能限制 (Feature Limits) - 拒绝过度膨胀

支持函数

复杂结构

取代 C#

仅支持数字、布尔值和文本

核心法则：开发者职责是搭建脚手架后“退避三舍”，解决编剧的具体痛点，而非构建一门完备的系统级编程语言，以防编剧沦为全职程序员。

边界内的妥协：用基础语法模拟复杂结构

情境难题与系统限制



编剧需求：在《I Feel Fine》中，想要一个数组（Array）来追踪这三个角色的交谈队列状态。

系统限制：工具本身不支持数组，且开发者拒绝添加该功能。

灰盒化解法（Workaround）

```
<<if $spoke_to_annex == false>>
  Annex: 我在排队!
  <<set $spoke_to_annex = true>>
<<elseif $spoke_to_beef == false>>
  Beef Chief: 该我了。
  <<set $spoke_to_beef = true>>
<<else>>
  Chomsky: 汪!
<<endif>>
```

使用一系列极简的 **if/else** 语句，为每个角色分配独立的布尔变量，追踪其交谈状态并控制队列。

因果结论：正因为具备基础的控制流，尽管语法受限，编剧依然能通过逻辑组合完全独立地实现复杂交互。

从幕后到台前：更丰富的分支，更深度的体验

第一层：零成本创作

[WRITER'S CORE]

编剧摆脱连线UI束缚，
创作速度爆发。

第二层：叙事系统有机增长

无阻碍探索催生出动态好感度、
角色记忆等隐性机制。

第三层：玩家体验

玩家最终体验到远超预期的多维网状叙事、
高反应度世界，以及高度个性化的多结局走向。

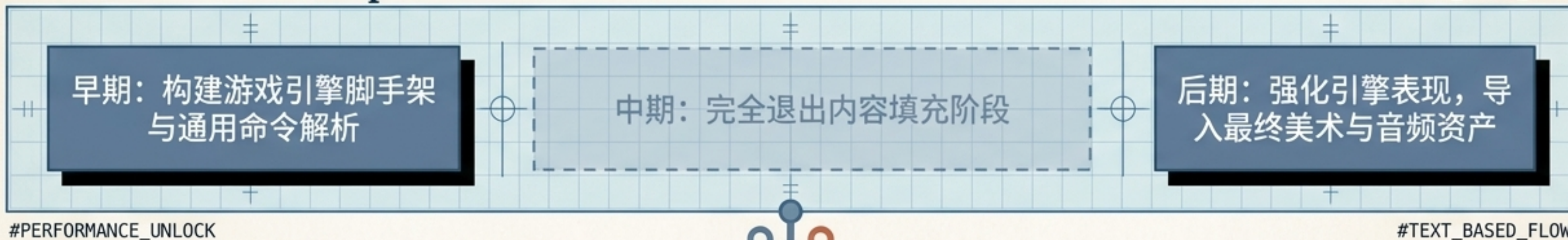
宏观把控外挂 [MACRO_CONTROL_TOOL]



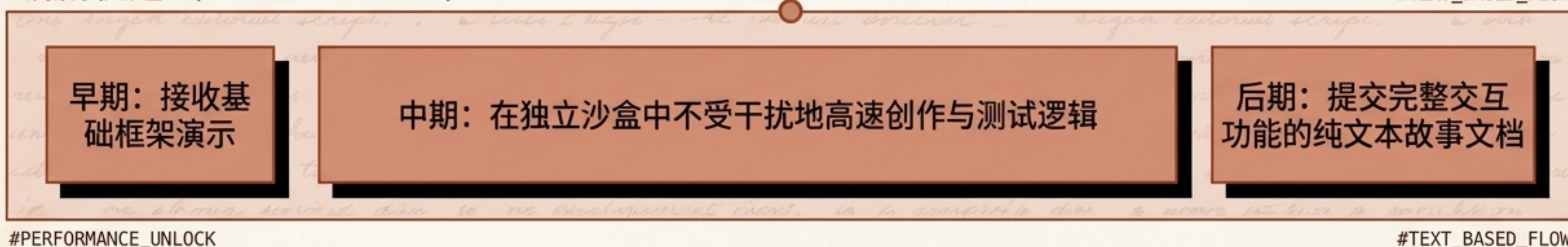
编译器级别的文本解析，
允许开发团队对对话树
进行基础块分析 (Basic
Block Analysis) 和控制
流图分析，全局监控故
事节奏与复杂度。

效能解绑：让编剧与开发者回归核心优势

开发者轨道 (Developer Track)



编剧轨道 (Writer Track)



工程附加值：由于一切产物皆为纯文本文件，传统的版本控制软件（Git/SVN）能完美追踪团队协作与历史更改。传统编剧的非线性创作门槛被降至最低。

工具的边界：领域特定语言的适用范围

不是什么 (What It's Not - 边界外)

- 绝不适合存储应放在“电子表格”中的非线性硬编码数据（如大量 RPG 物品属性）。
- 强行处理极其复杂的条件反射逻辑时，会导致灾难性的“巨型 if-else 树”，极难维护。

是什么 (What It Is - 核心定位)

- 它是一种专门针对“分支对话 (Branching Dialogue)”的领域特定语言 (DSL)。

最终结论：当游戏场景的“逻辑”就等同于场景的“故事”本身时（谁在做、谁在说），基于文本的故事语言能发挥出最大的威力。超出此边界的系统级数据交互，应交还给 C# 等专业工具。